

LAPORAN PRAKTIKUM
STRUKTUR DATA
QUEUE



OLEH:
DEVINA AMANDA PUTRI
(2411533009)

DOSEN PENGAMPU:
DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2024

A. PENDAHULUAN

Queue atau antrian adalah salah satu struktur data linear yang menerapkan prinsip First In First Out (FIFO), artinya elemen yang pertama kali dimasukkan akan menjadi elemen pertama yang dikeluarkan. Dalam implementasinya, struktur data queue dapat dianalogikan seperti barisan antrian loket tiket, dimana yang datang terlebih dahulu akan dilayani dahulu. Operasi dasar yang biasa dilakukan pada queue adalah enqueue atau menambah elemen ke belakang antrian, dan dequeue atau menghapus elemen dari depan antrian, serta operasi tambahan seperti melihat elemen terdepan (peek/front) atau terakhir (rear).

Queue dapat diimplementasikan menggunakan array, linkedlist maupun struktur yang disediakan oleh Java seperti, LinkedList. Dalam penggunaan praktisnya, queue sering diaplikasikan dalam sistem antrian data seperti buffer cetak, pemrosesan data real-time, penjadwalan proses dalam sistem operasi dan lainnya. Kemampuan queue untuk mengelola data secara teratur dan efisien sangat penting dalam banyak algoritma dan sistem berbasis komputasi

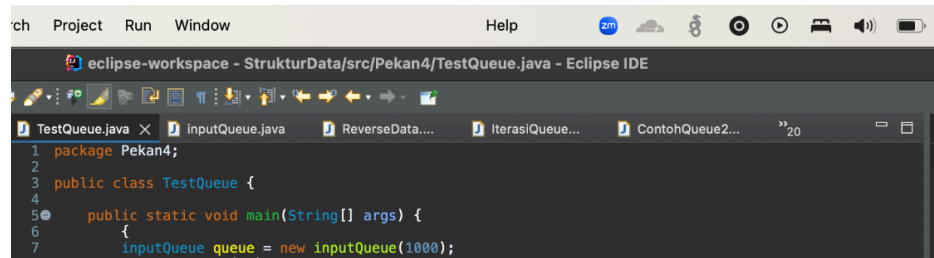
B. TUJUAN

1. Memahami konsep dasar struktur data queue dan prinsip FIFO
2. Mengimplementasikan queue menggunakan array
3. Mampu mengolah data dalam queue, termasuk iterasi dan pembalikan urutan

C. LANGKAH-LANGKAH Pengerjaan

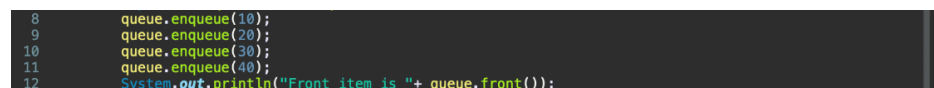
a. Class TestQueue.java

1. Buat class baru dan package Pekan4, membuat objek queue dari class inputQueue dengan kapasitas maksimum 1000



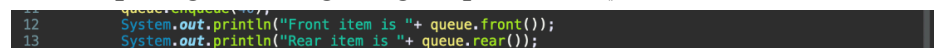
```
1 package Pekan4;
2
3 public class TestQueue {
4
5     public static void main(String[] args) {
6         inputQueue queue = new inputQueue(1000);
7     }
8 }
```

2. Menambahkan elemen ke dalam antrian menggunakan metode enqueue()



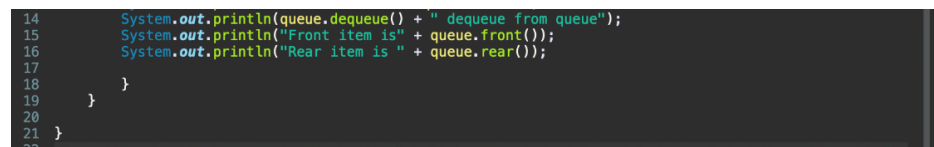
```
8 queue.enqueue(10);
9 queue.enqueue(20);
10 queue.enqueue(30);
11 queue.enqueue(40);
12 System.out.println("Front item is " + queue.front());
```

3. Menampilkan elemen paling depan dengan queue.front() dan elemen paling belakang dengan queue.rear()



```
12 System.out.println("Front item is " + queue.front());
13 System.out.println("Rear item is " + queue.rear());
```

4. Menghapus elemen paling depan pada antrian dengan queue.dequeue() lalu menampilkan queue setelah di dequeue. Menampilkan kembali elemen terdepan dan paling belakang setelah dihapus

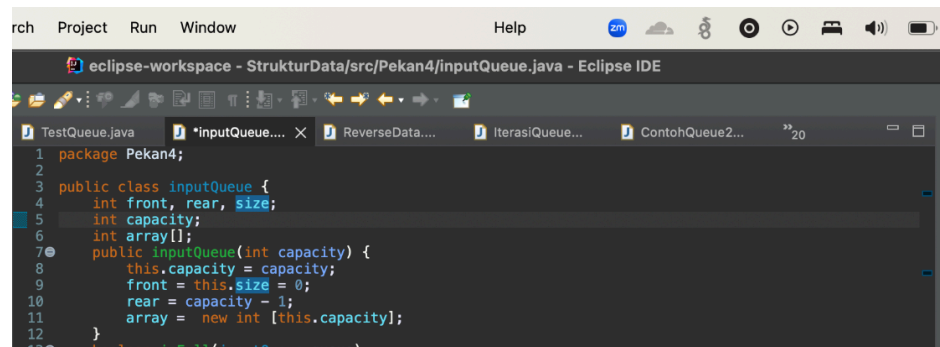


```
14 System.out.println(queue.dequeue() + " dequeue from queue");
15 System.out.println("Front item is " + queue.front());
16 System.out.println("Rear item is " + queue.rear());
17 }
18 }
19 }
20 }
21 }
22 }
```

b. Class inputQueue.java

1. Buat class baru dengan nama inputQueue pada package Pekan4. Deklarasikan variabel penting untuk mengatur posisi depan, belakang, ukuran dan kapasitas queue serta array penampung data. Tambahkan konstruktor untuk menginisialisasi queue sesuai

kapasitas input



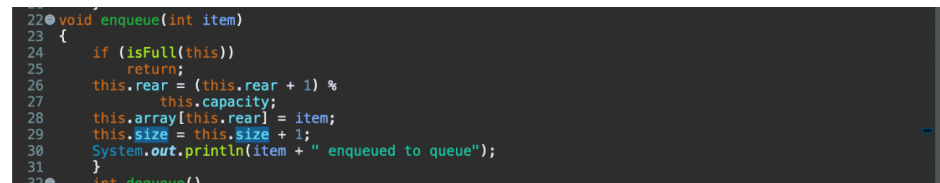
```
1 package Pekan4;
2
3 public class inputQueue {
4     int front, rear, size;
5     int capacity;
6     int array[];
7     public inputQueue(int capacity) {
8         this.capacity = capacity;
9         front = this.size = 0;
10        rear = capacity - 1;
11        array = new int [this.capacity];
12    }
13 }
```

2. Memeriksa apakah queue sudah penuh menggunakan isFull, lalu memeriksa apakah queue kosong menggunakan isEmpty



```
13 boolean isFull(inputQueue queue)
14 {
15     return (queue.size == queue.capacity);
16 }
17
18 boolean isEmpty(inputQueue queue)
19 {
20     return (queue.size == 0);
21 }
```

3. Menambahkan elemen ke dalam antrian, jika sudah penuh tidak menambahkan apapun.



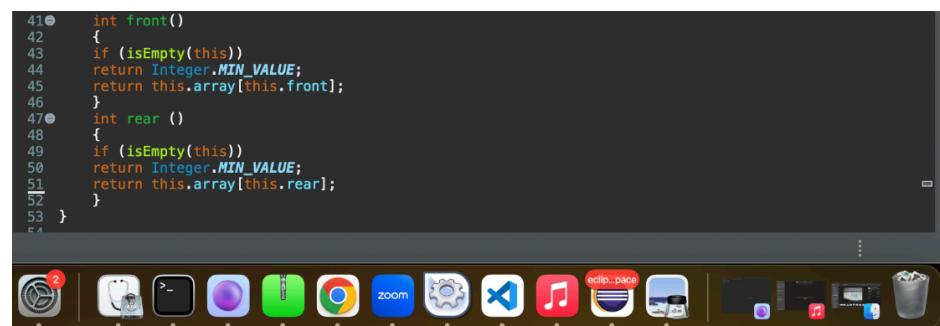
```
22 void enqueue(int item)
23 {
24     if (isFull(this))
25         return;
26     this.rear = (this.rear + 1) %
27         this.capacity;
28     this.array[this.rear] = item;
29     this.size = this.size + 1;
30     System.out.println(item + " enqueued to queue");
31 }
32 int dequeue()
```

4. Menghapus elemen pada method dequeue dari depan dan mengembalikannya lagi



```
32 int dequeue()
33 {
34     if (isEmpty(this))
35         return Integer.MIN_VALUE;
36     int item = this.array[this.front];
37     this.front = (this.front + 1) % this.capacity;
38     this.size = this.size - 1;
39     return item;
40 }
```

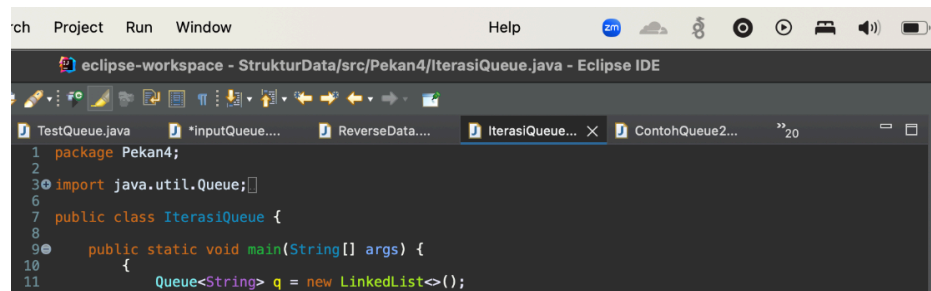
5. Mengembalikan elemen paling depan tanpa menghapusnya pada method front lalu mengembalikan elemen paling belakang pada method rear



```
41 int front()
42 {
43     if (isEmpty(this))
44         return Integer.MIN_VALUE;
45     return this.array[this.front];
46 }
47 int rear ()
48 {
49     if (isEmpty(this))
50         return Integer.MIN_VALUE;
51     return this.array[this.rear];
52 }
53 }
```

c. Class IterasiQueue.java

1. Buat class baru dengan nama IterasiQueue pada package yang sama, buat queue bertipe string menggunakan linkedlist



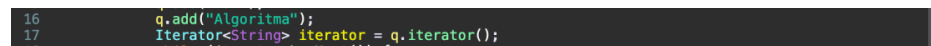
```
1 package Pekan4;
2
3 import java.util.Queue;
4
5
6
7 public class IterasiQueue {
8
9     public static void main(String[] args) {
10         {
11             Queue<String> q = new LinkedList<>();
```

2. Menambahkan beberapa elemen ke dalam queue dengan q.add



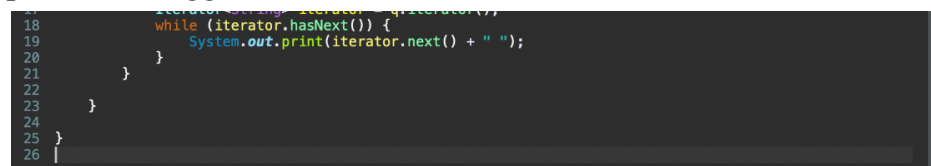
```
12         q.add("Praktikum");
13         q.add("Struktur");
14         q.add("Dan");
15         q.add("Algoritma");
```

3. Lalu membuat objek iterator untuk menelusuri atau mengecek satu per satu elemen dalam queue



```
16         q.add("Algoritma");
17         Iterator<String> iterator = q.iterator();
```

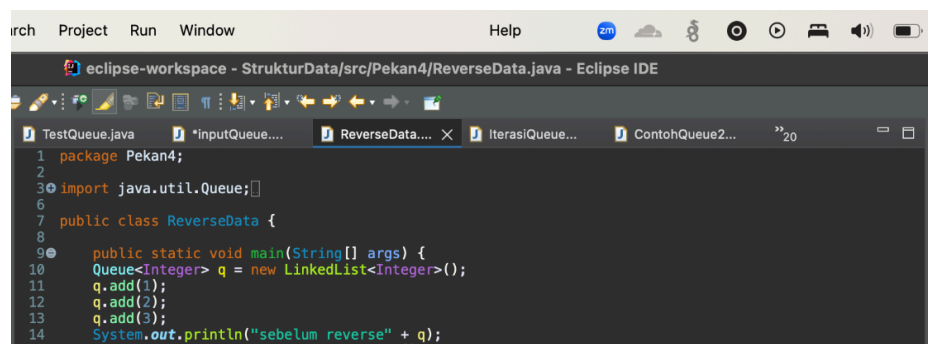
4. Mencetak seluruh elemen queue dengan perulangan while satu per satu menggunakan iterator



```
18         while (iterator.hasNext()) {
19             System.out.print(iterator.next() + " ");
20         }
21     }
22 }
23
24 }
25 }
26 }
```

d. Class ReverseData.java

1. Buat class baru dengan nama ReverseData pada package yang sama. Tambahkan objek Queue dengan LinkedList dan isi dengan angka 1,2, dan 3



```
1 package Pekan4;
2
3 import java.util.Queue;
4
5
6
7 public class ReverseData {
8
9     public static void main(String[] args) {
10         Queue<Integer> q = new LinkedList<Integer>();
11         q.add(1);
12         q.add(2);
13         q.add(3);
14         System.out.println("sebelum reverse" + q);
15         Stack<Integer> s = new Stack<Integer>();
```

2. Membuat stack s untuk menyimpan elemen sementara dari queue dan membantu membalik data



```
14         System.out.println("sebelum reverse" + q);
15         Stack<Integer> s = new Stack<Integer>();
16         while (!q.isEmpty());{
```

3. Dengan perulangan while, selama queue tidak kosong ambil elemen dari depan menggunakan q.remove() dan masukkan ke dalam stack, karena prinsip stack elemen pertama yang masuk, yaitu 1 akan berada paling bawah.

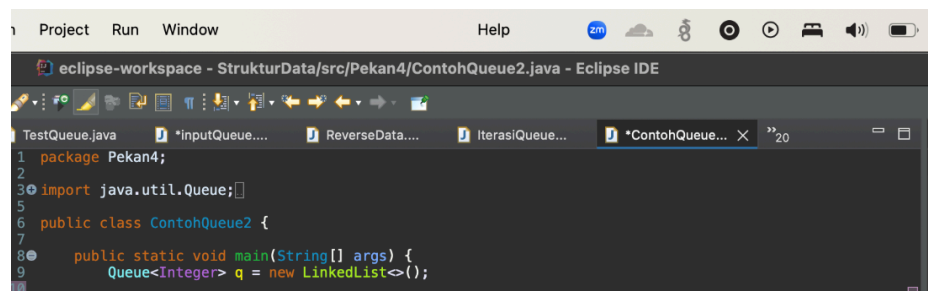
```
16 while (!q.isEmpty());{
17     s.push(q.remove());
18 }
19 while (!s.isEmpty()) {
20     q.add(s.pop());
21 }
```

4. Setelah stack penuh, elemen diambil kembali satu per satu dari atas dan dimasukkan ke dalam antrian lalu tampilkan isi antrian

```
19 while (!s.isEmpty()) {
20     q.add(s.pop());
21 }
22 System.out.println("sesudah reverse= " + q);
23 }
24 }
25 }
26 }
27 }
```

e. Class ContohQueue2.java

1. Buat class baru pada package yang sama, beri nama ContohQueue2.java, buat queue bertipe integer



The screenshot shows the Eclipse IDE interface. The top bar includes 'Project', 'Run', 'Window', and 'Help'. The main editor window is titled 'eclipse-workspace - StrukturData/src/Pekan4/ContohQueue2.java - Eclipse IDE'. The code in the editor is as follows:

```
1 package Pekan4;
2
3 import java.util.Queue;
4
5
6 public class ContohQueue2 {
7
8     public static void main(String[] args) {
9         Queue<Integer> q = new LinkedList<>();
10    }
```

2. Menambahkan angka 0 sampai 5 dengan perulangan for ke dalam queue dan menampilkan isi antrian

```
11 for (int i = 0; i < 6; i++) {
12     q.add(i);
13 }
14
15 System.out.println("Elemen Antrian = " + q);
16 int hapus = q.remove();
```

3. Menghapus elemen paling depan dan mencetak queue setelahnya lalu melihat elemen paling depan tanpa menghapusnya pada int depan dan menampilkan jumlah elemen dalam antrian

```
16 int hapus = q.remove();
17 System.out.println("Hapus elemen = " + hapus);
18 System.out.println(q);
19
20 int depan = q.peek();
21 System.out.println("Kepala Antrian = " + depan);
22
23
24 int banyak = q.size();
25 System.out.println("Size Antrian = " + banyak);
26 }
27
28 }
29 }
```

D. KESIMPULAN

Dari praktikum pekan 4 sudah dipelajari bagaimana cara kerja struktur data queue, baik secara manual menggunakan array maupun memanfaatkan Java Collection Framework. Sudah dipelajari pula cara melakukan iterasi yang berguna untuk mengecek setiap elemen dalam queue serta bagaimana cara membalik isi queue menggunakan bantuan stack.